

Epiplexity: information accessible to a bounded learner

Epiplexity with lightweight learning algorithms

Shannon entropy and Kolmogorov complexity do not explicitly restrict the computational resources of the observer. Finzi et al. (Finzi et al., 2026) introduce *epiplexity* as the structural information in a distribution that can be extracted by a computationally bounded observer. For a T -time probabilistic model P with program length $|P|$, define

$$P^* = \arg \min_{P \in \mathcal{P}_T} \{|P| + \mathbb{E}_X [-\log P(X)]\}.$$

The corresponding decomposition is

$$\underbrace{\text{MDL}_T(X)}_{\text{total time-bounded information}} = \underbrace{|P^*|}_{S_T(X) - \text{epiplexity}} + \underbrace{\mathbb{E}[-\log P^*(X)]}_{H_T(X) - \text{time-bounded entropy}}.$$

Thus, epiplexity is not simply a property of the dataset: it is defined relative to a computational budget and an admissible class of observers. In practice, neural training procedures provide tractable approximations to this program optimization. **Key reference:** Finzi et al., “From Entropy to Epiplexity”, 2026. The paper’s definition is compute-bounded; practical estimators approximate the optimal program through training trajectories.

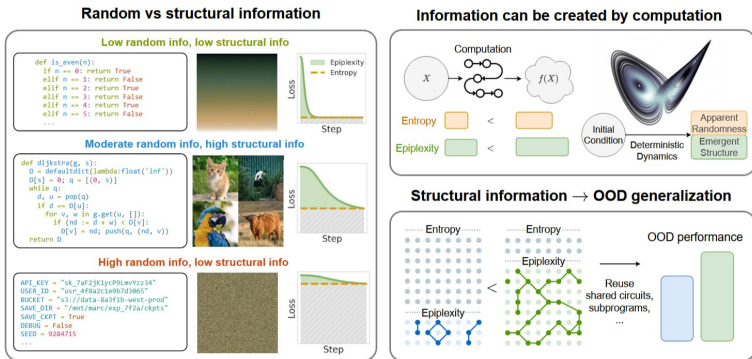


Figure 1: Random vs structural information for computationally bounded observers. (Left) Illustration of random vs structural information of different data for computationally bounded observers, which we formalize with time-bounded entropy and epiplexity (Section 3) and can be estimated from loss curves of neural networks trained on that data (Section 4). (Top Right) Unlike other forms of information, time-bounded entropy and epiplexity can be increased through computational processes, such as simulating dynamical systems (cellular automaton, Lorenz equations) and interventions like changing the data ordering, which can produce apparent randomness but also learnable, emergent structures like gliders and the Lorenz attractor invariant measure (Section 5). (Bottom Right) Whereas time-bounded entropy captures the in-distribution randomness and unpredictability, epiplexity measures the amount of structural information the model extracts from the data to its weights, which can be useful for OOD tasks such as by reusing learned circuits shared between the in-distribution and OOD tasks.

What we want to test: the estimators, not the architectures

The original empirical study demonstrates epiplexity with neural observers and sweeps over model size and training duration, including synthetic cellular automata and natural-data experiments (Finzi et al., 2026). Our question is not which architecture is better. We treat MLP, CNN and DiT as three different *setups* in which the same estimators are evaluated, and ask whether the estimated structural information remains comparable when the setup, the manipulations applied inside it, and the compute budget change:

$$X \xrightarrow{\text{setup } a} P_T, \quad a \in \{\text{MLP, CNN, DiT + sampler}\}.$$

Two things are varied at once and have to be told apart:

- the *manipulations* applied inside a setup (data ordering, pixel permutation, sampler schedule, budget);
- the *observer family* itself — hypothesis class and inductive bias.

The target is a statement of the form: this part of the estimate is a property of the data, and this part is produced by the setup in which we happened to measure it.

Estimators: one learning trajectory, several codes

We will evaluate several measures on the same training trajectories rather than introducing separate experimental setups for each metric.

Prequential coding. The description length is accumulated from predictions made by successive checkpoints before the corresponding training information is available. In the implementation of Finzi et al., the model-code contribution is estimated from the area of the training-loss curve above the final loss (Finzi et al., 2026):

$$K_{\text{preq}} \approx \sum_t \ell_t \Delta t \quad \text{relative to the final-loss baseline.}$$

Requential coding. Qiu et al. (Qiu et al., 2026) replace coding of the exact observed samples by coding synthetic samples on which teacher and student disagree. The central quantity is the cumulative divergence

$$K_{\text{req}} \propto \sum_t \text{KL}(P_t^{\text{teacher}} \parallel P_t^{\text{student}}).$$

This removes much of the cost associated with the entropy of the particular dataset sequence. **EDL.** Donoway et al. (Donoway et al., 2026) define Excess Description Length as the gap between sequential training-code length and the residual code length under the final predictor. It is closely related to learning dynamics, but is a measure for a specified learning algorithm rather than the optimization over all admissible programs defining epiplexity.

Experimental design: setups and the knobs attached to them

The dataset and the compute budget are fixed. What varies is the setup, its scale, and the manipulation applied inside it.

	MLP	CNN	DiT
Scale	10^3 – 10^6	10^3 – 10^6	small $\rightarrow$ medium
Prior	weakest	locality	generative, iterative
Optimizer	SGD / Adam(W)	SGD / Adam(W)	Adam(W)
Extra knob	input order	pixel permutation	sampler, #steps

The primary synthetic benchmark is elementary cellular automata, following the controlled setting of Finzi et al. (Finzi et al., 2026): their local transition rules give tasks with different degrees of learnable structure while keeping the data-generating mechanism explicit. Each setup carries its own manipulation, and the three are deliberately of a different kind:

- MLP: no built-in spatial prior, so the order in which inputs are presented is itself part of the observer;
- CNN: a fixed random permutation of pixels is applied to image data. Labels are unchanged, but the spatial arrangement the architecture exploits is destroyed;
- DiT: the sampler is part of the observer. Different schedules use different numbers of model evaluations and follow different trajectories through the learned score field.

Validation and readout

For every task d , setup a , compute budget T and seed s we record the learning trajectory and derive

$$S_{\text{preq}}, \quad S_{\text{req}}, \quad \text{EDL}.$$

Three checks are applied to the resulting table:

- **Agreement between estimators.** The orderings they assign to the same tasks are compared by Spearman correlation and Kendall's τ , both within and across setups.
- **Setup dependence.** For each pair of setups we report

$$\Delta_{ab}(d, T) = S(d, a, T) - S(d, b, T), \quad a, b \in \{\text{MLP}, \text{CNN}, \text{DiT}\}.$$

A large Δ with a stable task ordering means the estimator is partly re-measuring the setup rather than the data.

- **Seed variance.** Reported explicitly and compared against the spread across setups; final test loss and sample efficiency serve as external baselines.

The DiT setup is the sharpest case, because there the observer is not a single function but a compute schedule: we can ask whether this compute-bounded view separates sampling strategies from one another when final sample quality does not. The deliverable is a map of where these estimators are setup-invariant and where they are not — not a ranking of architectures.

Literature

- M. Finzi, S. Qiu, Y. Jiang, P. Izmailov, J. Zico Kolter, A. G. Wilson. *From Entropy to Epiplexity: Rethinking Information for Computationally Bounded Intelligence*. arXiv:2601.03220, 2026.
- S. Qiu, M. Finzi, Y. Zheng, K. Zhang, A. G. Wilson. *Requential Coding: Pushing the Limits of Model Compression with Self-Generated Training Data*. arXiv:2607.11883, 2026.
- E. Donoway, H. Joren, F. Roger, J. Leike. *Excess Description Length of Learning Generalizable Predictors*. arXiv:2601.04728, 2026.
- The project studies computationally bounded structural information through a controlled change of observer architecture, estimator, and compute budget.